Davante

7

ENTORNOS DE DESARROLLO

Elaboración de diagramas de clases

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Clases, atributos y métodos 4

/ 3. Visibilidad, objetos e instanciación 4

/ 4. Caso práctico 1: “Generación de diagrama a partir de código” 5

/ 5. Relación entre clases y elaboración de diagramas 6

/ 6. Creación de código a partir de diagramas de clase e ingeniería inversa 7

/ 7. Caso práctico 2: “Modelado de clases” 8

/ 8. Resumen y resolución del caso práctico de la unidad 9

/ 9. Bibliografía 9

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer el concepto de ingeniería inversa.

Saber utilizar los diagramas de clases.

Conocer herramientas de modelado.

Generación de códigos a partir de diagramas.

/ 1. Introducción y contextualización práctica

A la hora de desarrollar el software, es necesario pasar por una fase de diseño, en la que se utilizan diferentes tipos de diagramas. Uno de los diagramas más utilizados es el diagrama de clases, que permite representar las principales características de las clases de nuestro sistema. Además, a través de este diagrama, podemos realizar diferentes tareas, como la generación automática de código, facilitar que cualquier programador se pueda vincular al desarrollo de una manera más sencilla, etc.

Existen herramientas que permiten generar diagramas de clase de una manera muy sencilla, facilitando la tarea al diseñador o analista.

Una de las tareas que también incluye la ingeniería del software es la ingeniería inversa, que consiste en obtener el código fuente a partir de un código máquina. También es posible generar un diagrama de clases a partir de un código fuente.

Si llevamos todo al extremo, sería posible realizar una ingeniería inversa de una aplicación que, directamente, nos devuelva el diagrama de clases resultante. Aunque, técnicamente, esto es posible, en la realidad, los diagramas que generamos a través de la ingeniería inversa suelen ser poco precisos.



Fig. 1. Existen diferentes tipos de diagramas en UML.

Escucha el siguiente audio, en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio Intro. "Creación de diagramas de clase a partir de código"
https://bit.ly/30OhZ3b

/ 2. Clases, atributos y métodos

UML es un lenguaje estándar para el modelado de soluciones orientadas a objetos. Por ello, debemos conocer los principios básicos de la orientación a objetos para poder realizar el modelado correctamente.

Seguro que, de la asignatura de Programación, sabes que la orientación a objetos es un paradigma de desarrollo software cuyo principal elemento es el objeto. En este paradigma de programación, un objeto es un elemento que observamos en la vida real y que puede tener determinadas funcionalidades en su interior para realizar las tareas que queramos modelar. En este paradigma, es necesario disponer de clases para poder crear objetos.

Por tanto, una clase podrá definir el comportamiento, a nivel de código, que debe tener un objeto para, una vez sea instanciado, pueda ejecutar dicho código. Además de las funcionalidades, una clase puede tener diferentes atributos para definir o almacenar datos en su interior. Es importante diferenciar los datos que pertenecen a la clase y los datos que pertenecen al objeto.

Desde el punto de vista del diseño y modelado, podemos introducir el concepto de diagrama de clases. En la ingeniería del software, el diagrama de clase es un tipo de diagrama que permite describir la estructura de un sistema, utilizando las relaciones que existen entre las clases, junto con sus operaciones y atributos. Normalmente, el diagrama de clases se utiliza para realizar un modelo conceptual de la aplicación y trasladar todos esos detalles en una etapa posterior al código. Cuando visualizamos un diagrama de clase, podemos ver las principales relaciones e interacciones de las clases que forman parte de nuestra aplicación.

La forma de representar las clases se observa en la siguiente figura.

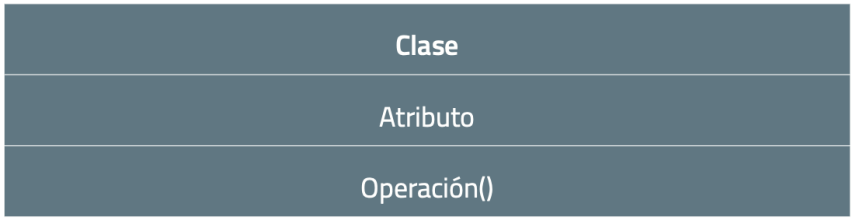


Fig. 2. Representación de una clase.

Vídeo 1. "Ejemplo de diagramas de clase"
https:/bit.ly/2ootKfN

/ 3. Visibilidad, objetos e instanciación

A. **Visibilidad**

En la programación orientada objetos, existe el concepto de **visibilidad**, tanto de los atributos como de los métodos que forman parte del objeto. Gracias a esto, podemos presentar el concepto de **encapsulación y ocultamiento de la información** dentro de una clase. Estos dos principios son sumamente importantes en la programación orientada objetos, pues permiten que el programador suba el nivel de abstracción y use las clases sin conocer los detalles de implementación.

Mediante la encapsulación, toda la funcionalidad de una clase queda dentro de dicha clase, sin tener que hacer referencia a clases externas.

El principio de ocultamiento de información trata de ocultar aquellas funcionalidades que no son necesarias para el programador pero que permiten la implementación de dicha clase. El ocultamiento de información está directamente relacionado con la visibilidad dentro de una clase.

Con la visibilidad, podemos modificar el nivel de detalle que permitimos visualizar. Normalmente, tenemos tres niveles de visibilidad:

- Público (visible para todos)
- Protegido (visible solo para las clases derivadas)
- Privado (visible solo dentro de la clase).

B. Objetos e instanciación

Como hemos visto anteriormente, para poder utilizar una clase, es necesario crear una instancia para crear un objeto. A través del proceso de instanciación, podemos crear objetos que van a seguir la estructura que está definida dentro de la clase que estamos utilizando.

Normalmente, para crear un objeto, se utiliza la palabra reservada new de lenguaje de programación con el que estemos trabajando. En los lenguajes de programación modernos, se tiende a suprimir la palabra reservada new para crear los objetos. A continuación, se muestra un ejemplo de cómo crear diferentes instancias de objetos siguiendo la sintaxis de Java:

```
1.Coche miCoche = new Coche();
2.Coche miCoche= new Coche("Rojo")
```

Código 1. Creación de instancias a objetos.



Audio 1. "Lenguajes modernos y objetos"
https://bit.ly/3iBMf7x



/ 4. Caso práctico 1: "Generación de diagrama a partir de código"

Planteamiento: En nuestro equipo, se nos plantea que tenemos que documentar rápidamente la aplicación que hemos desarrollado en Java, pues el cliente nos está pidiendo un diagrama de clases para ver cómo hemos estructurado el desarrollo; además, lo necesitaremos como base de cara a futuras ampliaciones o evoluciones del aplicativo.

Este proceso nos ayudará a saber, también, el estado en el que se encuentra nuestra aplicación.

Nudo: ¿Cómo llevarías esto a cabo? ¿Existe algún proceso para automatizarlo? ¿Crees que es correcto realizarlo a posteriori?

Desenlace: Según hemos visto a lo largo de la asignatura, es importante que creemos el diseño de nuestra aplicación y su modelado antes de pasar a la implementación.

Normalmente, las empresas tienden a obviar las fases de diseño y documentación del software, saltándose las etapas de modelado. Por ello, es cada vez más común observar que se genera la documentación una vez el software ya está generado. Aunque existen herramientas para generar el diagrama de clases a partir del código fuente, no es una buena práctica. El lenguaje UML, así como el diagrama de clases, se deben utilizar para modelar el software, y no generar el diagrama a partir del código que ya tenemos. Tenemos que utilizar el lenguaje UML para realizar el modelado y solventar problemas de dependencias, diseño, etc., antes de escribir cualquier línea de código. Si lo que hacemos es generar documentación, estamos usando mal el lenguaje UML. Para solucionarlo rápidamente, podríamos utilizar un plugin del IDE que estemos utilizando para crear los diagramas de clase. A continuación, se muestra una captura de Eclipse, en el que se permite generar el diagrama de clases a partir del código fuente.

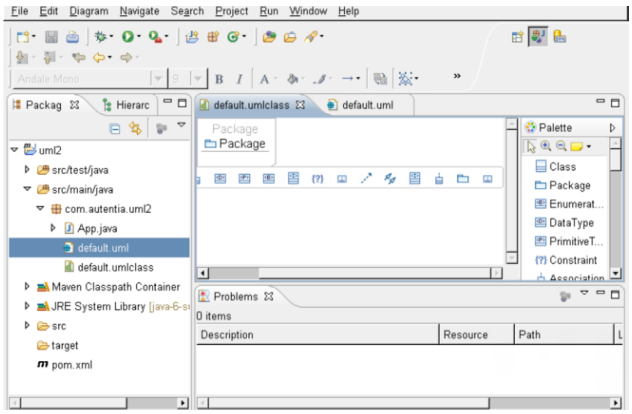


Fig. 3. Generación de diagrama a partir de código.

/ 5. Relación entre clases y elaboración de diagramas

A. **Relación entre clases**

El diagrama de clases, además de representar las clases que forman nuestro sistema, también muestra la relación que existe entre cada una de ellas para poder llevar a cabo la solución que estamos implementando. Las relaciones en un diagrama de clase se representan mediante una flecha que señalar la cardinalidad. Nótese que la cardinalidad permite indicar el número de elementos con el que se representa nuestra clase en una relación. Se puede utilizar un valor comodín para mostrar que ese número es indeterminado. Existen diferentes formas de relación que pueden tener las clases en nuestro sistema:

- **Asociación.** Es la relación más sencilla que podemos encontrar un diagrama de clase. No posee un tipo definido y se representa mediante una flecha simple.
- **Composición.** Permite definir las clases que pueden intervenir en nuestra clase para componerla. Se representa mediante una flecha con un rombo que apunta hacia la composición.
- **Dependencia.** Permite indicar que una clase usa otra, por lo que es necesario disponer de la primera para usar la nueva.
- **Herencia.** Permite establecer una jerarquía de clases. Se representa mediante una línea con una punta triangular vacía.

B. **Elaboración de diagramas de clases con CASE**

Como el diagrama de clases es uno de los más utilizados en el lenguaje UML, existe un gran número de herramientas que pueden facilitar la creación de este tipo de diagramas. Algunas de ellas son:

» **Microsoft Visio.** Se trata de una herramienta de propósito general que está diseñada para crear cualquier tipo de diagrama, entre otros, los UML.

» **Rational Rose.** Es una herramienta profesional, de pago, que está diseñada para seguir el proceso unificado y, por lo tanto, incluye todos los diagramas de lenguaje de modelado UML.

» **StarUML.** Se trata de una herramienta que permite generar diagramas UML de una manera sencilla. Puede ser utilizada de manera gratuita; además, puede añadirse como plugin en diferentes entornos de desarrollo.

Vídeo 2. "Herencia y asociación en diagramas de clase"
https://bit.ly/2ootYUb

/ 6. Creación de código a partir de diagramas de clase e ingeniería inversa

A. **Creación de código a partir de diagramas de clase**

Gracias a las herramientas que se ofrecen junto con los IDE, es posible generar código a partir de los diagramas de clase.

Si tenemos en cuenta cómo se representa un diagrama de clase, es relativamente sencillo generar código a partir de su representación.

Hay que tener en cuenta que el tipo de código que vamos a generar es genérico, y puede que no sea el más efectivo. Nótese que el código generado solo representa el modelo conceptual, pero en ningún caso implementará los métodos que darán funcionalidad a nuestra clase. Gracias a la generación automática, podemos ahorrar tiempo en la generación de dicho modelo; además, el diagrama de clase va a ser independiente del lenguaje final que estemos utilizando.

B. **Ingeniería inversa**

La ingeniería inversa es un campo del software que permite obtener el código fuente original a partir del código ejecutable o binario. Para poder aplicar estas técnicas, es necesario que el código sea **decompilado**.

Existen lenguajes que permiten una fácil decompilación, por ejemplo, Java. Sin embargo, existen otros en los que el proceso es más complejo, como sucede en C/C++.

Gracias a este proceso, es posible obtener el código fuente y generar una representación visual con el diagrama de clases correspondiente. Existen herramientas, como Eclipse y Visual Studio (entre otras), que permiten realizar todo este proceso en un único paso y de una manera bastante sencilla. Como desarrolladores, es necesario que sepamos que existen procesos para evitar que alguien obtenga fácilmente nuestro código fuente. Gracias a la ofuscación, es posible representar el código de tal forma que sea complicado obtener código original.



Fig. 4. La ingeniería inversa puede ser un preludio a un ataque software.

/ 7. Caso práctico 2: "Modelado de clases"

Planteamiento: Al ser el experto en ingeniería del software de nuestro equipo, nos piden que realicemos el modelado de un conjunto de clases a partir de la siguiente especificación:

- Clase Cuenta:
  - » Atributos:
    - balance: tipo int y privado
    - límite: tipo int y privado
    - fechaPublicación: tipo date y privado
  - » Métodos
    - Ingresar: devuelve int
    - Retirar: devuelve int
- Clase Estudiante:
  - » Atributos:
    - nombre: tipo string y privado
    - apellidos: tipo string y privado
    - nif: tipo string y privado
    - dirección: tipo string y privado

Nudo: ¿Cómo realizarías el diagrama de clases incluyendo sus relaciones? ¿Qué tipo de relación sería?

Desenlace: Para realizar el diagrama de clases, es necesario tener en cuenta la relación que existe entre la clase cuenta y estudiante. Como se puede observar, la relación existente es de asociación, y es posible que una cuenta se vincule con un estudiante, pero un estudiante puede tener, al menos, una cuenta (cardinalidad).

En la siguiente figura, se muestra el resultado de realizar el modelado del diagrama de clases. Existe una relación de asociación entre la clase cuenta y estudiante, en la cual indicamos la cardinalidad de ambas. Nótese que la cardinalidad va cruzada, de tal forma que, en el lado del estudiante, aparecerá el número de cuentas que puede tener, y viceversa.

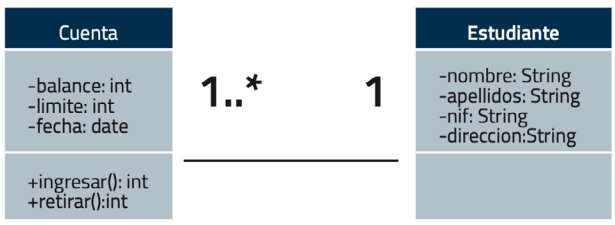


Fig. 5. Posible solución en un diagrama de clases mediante asociación.

/ 8. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema, hemos presentado las diferentes **características que intervienen en el diseño de una clase**, teniendo en cuenta su modelado utilizando UML. La utilización del diagrama de clases nos permite tener una representación visual de toda la relación que tienen las clases de nuestro diseño.

A lo largo del tema, también hemos estudiados conceptos como la **ingeniería inversa**, que permite obtener el código fuente a partir del código binario. La existencia de herramientas que permiten generar diagramas de clases facilita, en gran medida, la utilización del lenguaje UML. Además, algunas de estas herramientas incluyen la funcionalidad de generar código a partir del diagrama de clases. En la actualidad, una funcionalidad muy utilizada es la generación de diagramas a partir del código fuente, siendo esto parte de la ingeniería inversa.

Resolución del caso práctico de la unidad

Para resolver este caso de estudio, es necesario que nos trasladen lo que ya está creado.

Sabemos que ha sido creado en Java, lo cual, en este caso, no es relevante a la hora de crear el diagrama de clases, ya que este es independiente del lenguaje utilizado.

Sabemos que disponemos de dos clases, cada una de las cuales posee los atributos que se indican en el enunciado. De esta forma, el resultado del diagrama de clases se muestra a continuación. En el caso de la cardinalidad, hay que recordar que esta va cruzada.

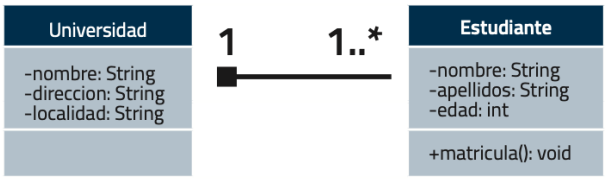


Fig. 6. Diagrama de clases que soluciona el caso.

/ 9. Bibliografía

Eilam, E. y Chikofsky, E. (2005). *Reversing: Secrets of reverse engineering.* Indianapolis: Wiley.

Juan, J. y Resusta, L. (2014). *UML práctico: aprende UML paso a paso.* España.

Pender, T., McSheffrey, E. y Varveris, L. (2003). *UML Bible.* Indianapolis: Wiley Pub.